SPEC-DRIVEN DEVELOPMENT · BMAD METHOD

End-to-End Exercise

Full Four-Agent BMad Workflow on the Spec-Driven MCP Server

AUTHOR	VERSION	DATE	FEATURE EXAMPLE
ahmed elpannann	2.0	May 2026	Password Reset Flow

Making Bob Think Before It Codes

Most AI coding assistants — including Bob — are reactive by default. You describe a feature and they immediately generate code. Without documented requirements, a technical design, or a task breakdown, the AI has no shared understanding of what it is building, why specific decisions were made, or how the work connects to the broader system. The result is code that is often correct in isolation but misaligned with the product, difficult to review, and expensive to change.

This workflow introduces a **Spec-Driven MCP server** layered with the **BMad-Method** agent framework, running entirely inside Bob. Four specialist agent personas guide the user through a structured three-phase workflow — Requirements, Design, Tasks — before any implementation begins. Every decision is persisted as a versioned Markdown file in the project repository, producing a complete, auditable engineering record alongside the code.

Workflow Architecture

The workflow operates across four layers and four phases:

- → **You (Developer):** Describe the feature, review briefs, approve designs, and review code.
- → **BMad Agents:** Four personas (Mary, Winston, John, Amelia) guide the process.
- → **MCP Server:** Provides 14 tools to validate EARS notation, store ADRs, assign Task IDs, and track status.
- → **Project Repository:** Stores `requirements.md` , `design.md` , `tasks.md` , and the working code.

What This Exercise Covers

This document walks through the complete BMad agent workflow — from initial idea to implemented code — using a realistic feature: a **User Password Reset Flow**. You will see exactly what to type, how each agent responds, and which MCP tools are called at each step.

Phase	Agent	Persona	Primary MCP Tool	Output
1 — Analysis	Mary	Business Analyst	`write_requirements`	requirements.md
2 — Architecture	Winston	Solutions Architect	`write_design`	design.md
3 — Story Breakdown	John	Product Manager	`write_tasks`	tasks.md
4 — Implementation	Amelia	Senior Developer	`update_task_status`	Working code

Prerequisites

Before starting this exercise, ensure you have the following in place.

Requirement	Detail
MCP Server Running	The Spec-Driven MCP server must be connected to your AI assistant (Cline, Cursor, or Bob).

Requirement	Detail
BMad Skills Installed	The `.bob/skills/` directory must be present in your project root.
Mode Active	Set your assistant to `bmad-spec-architect` mode, or instruct it to read the skills directory.
Project Initialised	You have a project folder open in VS Code with the MCP config pointing to the server.

How to Read This Document

Blue boxes are things **you type**. Teal boxes are **agent responses**. Purple boxes are **MCP tool calls**. Orange boxes are **notes**.

The Feature We Are Building

We will build a **Password Reset Flow** — a common, well-understood feature that is rich enough to exercise all four agents. The feature allows users to securely reset their password via an email link, using the existing SendGrid integration, with tokens that expire after 15 minutes.

SETUP

Configuring Bob for the BMad + Spec-Driven Workflow

Install the MCP server · Load the mode · Populate steering files · Launch Bob

Before running the four-agent workflow, Bob must be configured to connect to the Spec-Driven MCP server and load the BMad orchestrator mode. The repository already contains all required configuration files in the `.bob/` directory — you only need to follow the steps below once per project.

STEP 0.1

Verify Prerequisites

Ensure the following are installed and available on your system before proceeding.

Requirement	Check	Install Command
Python 3.11+	`python --version`	`winget install Python.Python.3.11`
uv / uvx	`uvx --version`	`pip install uv`
fastmcp	`uvx fastmcp --version`	Installed automatically by uvx on first run
Bob (VS Code extension)	Extensions panel → search "Bob"	Install from VS Code Marketplace

STEP 0.2

Open the Project Folder in VS Code

YOU

Open VS Code and use **File → Open Folder** to open the root of the `universal-spec-mcp` repository. The `.bob/` directory must be at the project root for Bob to detect it automatically.

NOTE

Bob detects the `.bob/mcp.json` file on startup and launches the MCP server automatically using the command: `uvx fastmcp run src/universal_spec_mcp/server.py` . No manual server startup is required.

STEP 0.3

Launch Bob

YOU

Open the Bob application, then double-click the `.bat` file to launch it. Bob will initialise and connect to the MCP server defined in `.bob/mcp.json` .

NOTE — .BOB/MCP.JSON

```
{
  "mcpServers": {
```

```
    "universal-spec-architect": {
      "command": "uvx",
      "args": ["fastmcp", "run", "src/universal_spec_mcp/server.py"],
      "env": { "PYTHONPATH": "src" },
      "timeout": 300
    }
  }
}
```

STEP 0.4

Select the BMad Spec Architect Mode

YOU

In Bob's UI, open the mode selector and choose **BMad Spec Architect**. This mode is defined in `.bob/modes/bmad-spec-architect.json` and automatically loads all four agent skills (bmad-analyst, bmad-architect, bmad-pm, bmad-dev).

BOB — BMAD SPEC ARCHITECT MODE

BMad Spec Architect mode activated. I have loaded the following skills: bmad-analyst (Mary), bmad-architect (Winston), bmad-pm (John), bmad-dev (Amelia). The MCP server is connected. Type the name of a skill to activate an agent, or type `bmad-help` to assess the current project state.

STEP 0.5

Populate the Steering Files

Bob reads three steering files before making any decisions. These files are in `.bob/steering/` and must be filled in for your project. Open each file and replace the placeholder text.

File	What to Write	Example
`product.md`	Product purpose, target users, key features, business objectives	"A B2B SaaS invoicing tool for freelancers. Target users: independent contractors. Key feature: automated invoice generation."
`tech.md`	Tech stack, frameworks, database, external services	"Python 3.11, FastAPI, PostgreSQL 15, SendGrid for email, Alembic for migrations."
`structure.md`	Folder layout, naming conventions, code organisation rules	"src/api/ for routes, src/services/ for business logic, src/models/ for SQLAlchemy models."

NOTE

If the steering files are empty, agents will still function but will produce generic output. Populated steering files ensure every agent's output is grounded in your project's actual constraints.

→ Configuration complete. Proceed to Phase 1 — activate Mary to begin analysis.

PHASE 1

Analysis — Mary (Business Analyst)

Clarify the problem · Define success · Produce EARS-notation requirements

Mary's role is to clarify the problem before any technical work begins. She does not suggest solutions — she asks structured questions to ensure the requirements are unambiguous and testable.

STEP 1.1

Activate Mary

YOU

Switch to the bmad-analyst skill. I want to build a password reset feature for our users.

MARY — BUSINESS ANALYST

Hello! I'm Mary. Before we write any requirements, I need to ask four questions:

1. What problem are we solving? Who experiences it?
2. What does success look like? How will we measure it?
3. What are the constraints (time, technology, compliance)?
4. What is explicitly out of scope?

STEP 1.2

Answer the Probing Questions

YOU

1. Users forget their passwords and get locked out.
2. Success is a user securely resetting their password via email without contacting support.
3. Must use our existing SendGrid email service. Tokens must expire in 15 minutes.
4. Out of scope: SMS/Phone reset, social login recovery.

STEP 1.3

Confirm the Product Brief

MARY — BUSINESS ANALYST

Product Brief:
Problem: Users who forget their passwords are locked out and require a self-service recovery mechanism.
Goals: Enable secure, self-service password resets via email without support intervention.
Constraints: Must use SendGrid; reset tokens must expire in 15 minutes.
Out of Scope: SMS recovery, social login recovery.

Does this look correct? If so, I will translate this into EARS-notation requirements.

YOU

Yes, looks good.

STEP 1.4

Write Requirements via MCP

NOTE

Mary calls `initialize_spec` first to create the spec folder, then `write_requirements` with EARS-notation statements. The MCP server validates each statement against the EARS grammar before saving.

MCP TOOL CALL — COMPLETE VALID PAYLOAD

```
initialize_spec("password-reset", "requirements-first")

write_requirements("password-reset", {
  "feature_name": "password-reset",
  "description": "Self-service password reset via email using SendGrid. Tokens expire
after 15 minutes.",
  "requirements": [
    {
      "id": "REQ-001",
      "title": "Request Reset Email",
      "ears_statement": "WHEN a user requests a password reset THE system SHALL send
an email containing a secure reset link",
      "acceptance_criteria": ["Email is received within 60 seconds", "Link contains a
unique one-time token"]
    },
    {
      "id": "REQ-002",
      "title": "Token Expiry Enforcement",
      "ears_statement": "WHILE a reset token is older than 15 minutes THE system SHALL
reject the reset attempt",
      "acceptance_criteria": ["Expired token returns HTTP 400", "Error message is
displayed to user"]
    },
    {
      "id": "REQ-003",
      "title": "Password Update on Valid Token",
      "ears_statement": "WHEN a user submits a valid reset token THE system SHALL
allow the user to set a new password",
      "acceptance_criteria": ["New password is saved", "Token is invalidated after
use", "User is redirected to login"]
    }
  ]
})
```

MARY — BUSINESS ANALYST

Requirements are locked and saved to `specs/password-reset/requirements.md` . All 3 requirements passed EARS validation.

Handing off to **Winston (Architect)** for technical design. Switch to the `bmad-architect` skill to continue.

→ Phase 1 complete. Switch to bmad-architect skill to begin Phase 2.

PHASE 2

Architecture — Winston (Solutions Architect)

Read requirements · Design the solution · Document ADRs · Save design

Winston reads Mary's requirements before proposing any design. He documents every significant technical decision as an Architecture Decision Record (ADR) so the rationale is preserved for future developers.

STEP 2.1

Activate Winston and Read Requirements

YOU

Switch to the bmad-architect skill.

MCP TOOL CALL

```
read_spec("password-reset", "requirements")
```

WINSTON — SOLUTIONS ARCHITECT

Hello, I'm Winston. I have read the requirements. I see we need a password reset flow using SendGrid with 15-minute token expiry. I will draft the architecture, including data models, a sequence diagram, and an ADR for how we store the reset token.

STEP 2.2

Review the Draft Design

WINSTON — SOLUTIONS ARCHITECT

Here is the proposed design:

Data Model: Add `reset_token_hash` (VARCHAR 64) and `reset_token_expires_at` (TIMESTAMP) to the `users` table.

ADR-001 — Token Storage: We will store a SHA-256 hash of the token in the database, not the plain-text token. This prevents token theft if the database is compromised. The plain-text token is only ever present in the email link and in memory during validation.

Sequence: User → POST /auth/reset-request → API generates token → SendGrid sends email → User clicks link → POST /auth/reset-confirm → API validates hash and expiry → Password updated.

Does this design address all requirements? Are there any constraints I missed?

YOU

The design looks solid. Go ahead and save it.

STEP 2.3

Write Design via MCP

MCP TOOL CALL — COMPLETE VALID PAYLOAD

```
write_design("password-reset", {
  "feature_name": "password-reset",
  "sections": [
    {
      "title": "Data Model",
      "content": "Add reset_token_hash (VARCHAR 64) and reset_token_expires_at
(TIMESTAMP) to the users table. Both columns are nullable to avoid breaking existing
rows."
    },
    {
      "title": "ADR-001: Token Storage Strategy",
      "content": "Decision: store SHA-256 hash of the token, not the plain-text value.
Rationale: if the database is compromised, the attacker cannot use the stored value to
reset passwords. The plain-text token exists only in the email link and in memory
during the single validation call."
    },
    {
      "title": "Sequence Diagram",
      "content": "User → POST /auth/reset-request → API generates token → SHA-256 hash
stored → SendGrid sends email → User clicks link → POST /auth/reset-confirm → API
hashes submitted token → compare with DB → if match and not expired → update password
→ invalidate token"
    },
    {
      "title": "Error Handling",
      "content": "Expired token: HTTP 400 with message 'Reset link has expired'.
Invalid token: HTTP 400 with message 'Invalid reset link'. User not found: HTTP 200
silent success (anti-enumeration — do not reveal whether the email exists)."
    }
  ]
})
```

WINSTON — SOLUTIONS ARCHITECT

Design is locked and saved to `specs/password-reset/design.md`.

Handing off to **John (Product Manager)** to break this into implementable stories. Switch to the `bmad-pm` skill to continue.

→ Phase 2 complete. Switch to bmad-pm skill to begin Phase 3.

PHASE 3

Story Breakdown — John (Product Manager)

Read requirements + design · Break into tasks · Define acceptance criteria · Save tasks

John reads both the requirements and design documents before proposing tasks. Each task must be independently deliverable, have clear acceptance criteria, and declare its dependencies explicitly.

STEP 3.1

Activate John and Read Spec Documents

YOU

Switch to the bmad-pm skill.

MCP TOOL CALLS

```
read_spec("password-reset", "requirements")
read_spec("password-reset", "design")
```

JOHN — PRODUCT MANAGER

Hello, I'm John. I have read both documents. I will break this down into discrete tasks, each taking roughly 1–4 hours with clear acceptance criteria.

STEP 3.2

Review the Proposed Tasks

JOHN — PRODUCT MANAGER

Here are the proposed tasks:

TASK-001 — DB Migration: Add `reset_token_hash` and `reset_token_expires_at` to the users table. No dependencies.
TASK-002 — Request Reset Endpoint: POST /auth/reset-request — generate token, hash it, store it, trigger SendGrid. Depends on TASK-001.
TASK-003 — SendGrid Integration: Send the reset email template via SendGrid with the plain-text token in the link. Depends on TASK-002.
TASK-004 — Confirm Reset Endpoint: POST /auth/reset-confirm — validate token hash and expiry, update password. Depends on TASK-001.

Are there any tasks missing? Is the priority order correct?

YOU

Looks perfect. Save the tasks.

STEP 3.3

Write Tasks via MCP

MCP TOOL CALL — COMPLETE VALID PAYLOAD

```
write_tasks("password-reset", {
  "feature_name": "password-reset",
  "tasks": [
    {
      "id": "TASK-001",
      "title": "DB Migration: add reset token columns",
      "description": "Write and apply an Alembic migration that adds reset_token_hash
(VARCHAR 64, nullable) and reset_token_expires_at (TIMESTAMP, nullable) to the users
table.",
      "status": "todo",
      "dependencies": [],
      "acceptance_criteria": ["Migration runs without error", "Rollback also works
cleanly", "Existing user rows are unaffected"]
    },
    {
      "id": "TASK-002",
      "title": "POST /auth/reset-request endpoint",
      "description": "Generate a cryptographically secure token, store its SHA-256
hash and a 15-minute expiry on the user record, then call the SendGrid service to
dispatch the reset email.",
      "status": "todo",
      "dependencies": ["TASK-001"],
      "acceptance_criteria": ["Returns HTTP 200 regardless of whether email exists
(anti-enumeration)", "Token hash and expiry are persisted to DB", "SendGrid is called
with correct template"]
    },
    {
      "id": "TASK-003",
      "title": "SendGrid reset email template",
      "description": "Create the SendGrid dynamic template for the password reset
email. The plain-text token must be embedded in the reset link URL as a query
parameter.",
      "status": "todo",
      "dependencies": ["TASK-002"],
      "acceptance_criteria": ["Email renders correctly in Gmail and Outlook", "Reset
link navigates to the correct confirm page", "Template ID is stored in environment
config, not hardcoded"]
    },
    {
      "id": "TASK-004",
      "title": "POST /auth/reset-confirm endpoint",
      "description": "Accept the plain-text token and new password. Hash the token,
compare with DB, check expiry, update password with bcrypt hash, and invalidate the
token by nulling both reset columns.",
      "status": "todo",
      "dependencies": ["TASK-001"],
      "acceptance_criteria": ["Expired token returns HTTP 400", "Invalid token returns
HTTP 400", "Valid token updates password and nulls reset columns", "Token cannot be
reused after successful reset"]
    }
  ]
})
```

JOHN — PRODUCT MANAGER

Stories are locked and saved to `specs/password-reset/tasks.md`. All 4 tasks are marked as `TODO`.

Handing off to **Amelia (Developer)** for implementation. Switch to the `bmad-dev` skill to continue.

→ Phase 3 complete. Switch to bmad-dev skill to begin Phase 4.

PHASE 4

Implementation — Amelia (Senior Developer)

Read tasks · Implement task by task · Track status via MCP · Verify completion

Amelia reads all spec documents before writing any code. She works through tasks in dependency order, updating the MCP status at the start and end of each task so progress is always visible in the spec files.

STEP 4.0

Activate Amelia — Survey the Project First

YOU

Switch to the bmad-dev skill.

MCP TOOL CALL

```
list_specs()
```

AMELIA — SENIOR DEVELOPER

Hello, I'm Amelia. I can see the following specs in this project:

password-reset — status: tasks_written — 3 requirements, 1 design document, 4 tasks (all todo)

I will now read all three documents for `password-reset` before writing any code.

STEP 4.1

Read All Spec Documents

MCP TOOL CALLS

```
read_spec("password-reset", "requirements")
read_spec("password-reset", "design")
read_spec("password-reset", "tasks")
```

AMELIA — SENIOR DEVELOPER

Hello, I'm Amelia. I have read all spec documents. I see 4 tasks marked as `TODO`. I will start with TASK-001 (DB Migration) as it has no dependencies.

STEP 4.2

Implement TASK-001 — DB Migration

MCP TOOL CALLS

```
run_hook("pre_task", "Starting TASK-001: DB Migration")
update_task_status("password-reset", "TASK-001", "in_progress")
```

AMELIA — SENIOR DEVELOPER

Writing the Alembic migration to add `reset_token_hash` and `reset_token_expires_at` to the users table...

```
def upgrade():
    op.add_column('users',
        sa.Column('reset_token_hash', sa.String(64), nullable=True))
    op.add_column('users',
        sa.Column('reset_token_expires_at', sa.DateTime, nullable=True))
```

MCP TOOL CALLS

```
update_task_status("password-reset", "TASK-001", "completed")
run_hook("post_task", "Completed TASK-001: DB Migration")
```

STEP 4.3

Implement TASK-002 — Request Reset Endpoint

MCP TOOL CALLS

```
run_hook("pre_task", "Starting TASK-002")
update_task_status("password-reset", "TASK-002", "in_progress")
```

```
@router.post("/auth/reset-request")
def request_reset(email: str, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.email == email).first()
    if user:
        token = secrets.token_urlsafe(32)
        user.reset_token_hash = hashlib.sha256(token.encode()).hexdigest()
        user.reset_token_expires_at = datetime.utcnow() + timedelta(minutes=15)
        db.commit()
        send_reset_email(email, token)   # TASK-003
    return {"message": "If that email exists, a reset link has been sent."}
```

MCP TOOL CALLS

```
update_task_status("password-reset", "TASK-002", "completed")
run_hook("post_task", "Completed TASK-002")
```

STEP 4.4

Implement TASK-003 and TASK-004

YOU

Go ahead and implement the remaining tasks.

AMELIA — SENIOR DEVELOPER

Implementing TASK-003 (SendGrid integration) and TASK-004 (confirm reset endpoint) using the same pre_task → in_progress → code → completed → post_task pattern...

MCP TOOL CALLS (REPEATED FOR EACH TASK)

```
run_hook("pre_task", "Starting TASK-003") → update_task_status("password-reset",
"TASK-003", "in_progress") → … → update_task_status("password-reset", "TASK-003",
"completed") → run_hook("post_task", "Completed TASK-003")

run_hook("pre_task", "Starting TASK-004") → update_task_status("password-reset",
```

"TASK-004", "in_progress") → … → update_task_status("password-reset", "TASK-004", "completed") → run_hook("post_task", "Completed TASK-004")

STEP 4.5

Verify Feature Complete

MCP TOOL CALL

```
read_spec("password-reset", "tasks")
```

AMELIA — SENIOR DEVELOPER

Feature implementation complete. All 4 tasks are marked as `COMPLETED` in `specs/password-reset/tasks.md`.

Consider switching to the `bmad-analyst` skill for the next feature.

→ Phase 4 complete. The password-reset feature is fully specced, designed, tasked, and implemented.

What Was Produced

File	Created By	Contents
`specs/password-reset/meta.json`	Mary (initialize_spec)	Feature name, workflow variant, status
`specs/password-reset/requirements.md`	Mary (write_requirements)	3 EARS-notation requirements with acceptance criteria
`specs/password-reset/design.md`	Winston (write_design)	Data model, ADR-001, sequence diagram, error handling
`specs/password-reset/tasks.md`	John (write_tasks)	4 tasks with dependencies and status tracking

Full MCP Tool Reference

Tool	Agent	Purpose
`initialize_spec`	Mary	Create spec folder and meta.json
`write_requirements`	Mary	Validate EARS and save requirements.md
`read_spec`	Winston, John, Amelia	Read requirements / design / tasks / meta
`write_design`	Winston	Save design.md with sections and ADRs
`write_tasks`	John	Save tasks.md with dependencies
`update_task_status`	Amelia	Set task to todo / in_progress / completed / blocked
`run_hook`	Amelia	Trigger pre_task / post_task lifecycle events
`list_specs`	Any	List all feature specs in the project
`search_specs`	Any	Full-text search across all spec documents
`add_directive`	Any	Add a project-wide rule injected into all responses
`remove_directive`	Any	Remove an active directive by ID
`list_directives`	Any	List all active directives
`add_memory`	Any	Store an architectural decision or context note
`get_memories_by_feature`	Any	Retrieve all memories for a specific feature

Task Status Markers in tasks.md

Status	Marker	Meaning
TODO	`[ ]`	Not yet started

Status	Marker	Meaning
IN_PROGRESS	[~]	Currently being implemented
COMPLETED	[x]	Done and verified
BLOCKED	[!]	Cannot proceed — dependency or external blocker

If You Get Lost

Type **"bmad-help"** at any point. The bmad-help skill will call `list_specs()` and `read_spec()` to assess your current state, then return a structured orientation report: which spec files exist, which are missing, which tasks are still pending, and a specific recommendation such as "requirements.md exists but design.md is missing — switch to bmad-architect and run write_design" or "TASK-002 is in_progress but TASK-001 is not yet completed — resolve the dependency before continuing".

After reading the orientation report, simply follow the recommended action: switch to the named skill and call the named tool. If the report says a document is missing, do not skip it — every downstream agent depends on the previous agent's output being present in the MCP server.

Key Principles of the BMad + Spec-Driven Workflow

- → No code is written until requirements, design, and tasks are locked in the MCP server.
- → Every agent reads the previous agent's output via `read_spec` before proceeding.
- → Every significant technical decision is documented as an ADR in the design document.
- → Task status is updated in real time via MCP — the spec files are always the source of truth.
- → The privacy filter automatically redacts secrets (API keys, tokens) from all saved spec files.